EXPERIMENT 2

Relational Schema and MySQL Implementation for a Campus Event Portal

1. Aim

To convert the ER model from Experiment 1 into relational tables, implement the tables in MySQL, add sample records, and test primary-key, foreign-key, uniqueness, check, cascade, and set-null rules.

2. Theory

A relational schema defines tables, columns, keys, and constraints. Each strong entity becomes a table. A one-to-many relationship is commonly implemented by placing the primary key of the parent on the child side as a foreign key. A many-to-many relationship is represented by a junction table whose key can be the combination of the two participating keys. A multivalued property is stored in a separate table so each value occupies its own row.

In this design, REGISTRATION connects STUDENT and CAMPUS_EVENT using a composite primary key (StudentID, EventID), which prevents a student from registering for the same event twice. PAYMENT and FEEDBACK reference that pair, ensuring they relate to a real registration. NOT NULL requires a value, UNIQUE prevents duplicate values, CHECK restricts values to a valid range, ON DELETE CASCADE removes dependent rows, and ON DELETE SET NULL preserves an event if its organizer or venue is removed.

3. Relational Schema

STUDENT(StudentID, FullName, Email, Phone, Department, Semester)
ORGANIZER(OrganizerID, OrganizationName, ContactEmail)
VENUE(VenueID, VenueName, Building, Capacity)
CAMPUS_EVENT(EventID, Title, EventDate, StartTime, EntryFee, EventStatus, OrganizerID, VenueID)
REGISTRATION(StudentID, EventID, RegisteredAt, AttendanceStatus)
PAYMENT(PaymentID, StudentID, EventID, Amount, PaymentStatus, PaymentDate)
FEEDBACK(FeedbackID, StudentID, EventID, Rating, Comments)
EVENT_TAG(EventID, TagName)

4. MySQL Table Creation

Execute the following script in MySQL Workbench or the MySQL command line. Create the database first, then run the table definitions in order so referenced parent tables exist before child tables.

```
CREATE DATABASE CampusEvents;
USE CampusEvents;

CREATE TABLE STUDENT (
    StudentID INT PRIMARY KEY,
    FullName VARCHAR(100) NOT NULL,
    Email VARCHAR(120) NOT NULL UNIQUE,
    Phone VARCHAR(15) UNIQUE,
    Department VARCHAR(60) NOT NULL,
    Semester TINYINT NOT NULL CHECK (Semester BETWEEN 1 AND 8)
);

CREATE TABLE ORGANIZER (
    OrganizerID INT PRIMARY KEY,
    OrganizationName VARCHAR(100) NOT NULL UNIQUE,
    ContactEmail VARCHAR(120) NOT NULL UNIQUE
);

CREATE TABLE VENUE (
    VenueID INT PRIMARY KEY,
    VenueName VARCHAR(100) NOT NULL,
    Building VARCHAR(80) NOT NULL,
    Capacity INT NOT NULL CHECK (Capacity > 0)
);

CREATE TABLE CAMPUS_EVENT (
    EventID INT PRIMARY KEY,
    Title VARCHAR(150) NOT NULL,
```

```
    EventDate DATE NOT NULL,
    StartTime TIME NOT NULL,
    EntryFee DECIMAL(8,2) NOT NULL DEFAULT 0 CHECK (EntryFee >= 0),
    EventStatus VARCHAR(20) NOT NULL DEFAULT 'Planned',
    OrganizerID INT NULL,
    VenueID INT NULL,
    FOREIGN KEY (OrganizerID) REFERENCES ORGANIZER(OrganizerID)
        ON DELETE SET NULL,
    FOREIGN KEY (VenueID) REFERENCES VENUE(VenueID)
        ON DELETE SET NULL
);

CREATE TABLE REGISTRATION (
    StudentID INT NOT NULL,
    EventID INT NOT NULL,
    RegisteredAt DATETIME NOT NULL,
    AttendanceStatus VARCHAR(20) NOT NULL DEFAULT 'Registered',
    PRIMARY KEY (StudentID, EventID),
    FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID)
        ON DELETE CASCADE,
    FOREIGN KEY (EventID) REFERENCES CAMPUS_EVENT(EventID)
        ON DELETE CASCADE
);

CREATE TABLE PAYMENT (
    PaymentID INT PRIMARY KEY,
    StudentID INT NOT NULL,
    EventID INT NOT NULL,
    Amount DECIMAL(8,2) NOT NULL CHECK (Amount >= 0),
    PaymentStatus VARCHAR(20) NOT NULL,
    PaymentDate DATE,
    FOREIGN KEY (StudentID, EventID)
        REFERENCES REGISTRATION(StudentID, EventID)
        ON DELETE CASCADE
);

CREATE TABLE FEEDBACK (
    FeedbackID INT PRIMARY KEY,
    StudentID INT NOT NULL,
    EventID INT NOT NULL,
    Rating TINYINT NOT NULL CHECK (Rating BETWEEN 1 AND 5),
    Comments VARCHAR(500),
    FOREIGN KEY (StudentID, EventID)
        REFERENCES REGISTRATION(StudentID, EventID)
        ON DELETE CASCADE
);

CREATE TABLE EVENT_TAG (
    EventID INT NOT NULL,
    TagName VARCHAR(40) NOT NULL,
    PRIMARY KEY (EventID, TagName),
    FOREIGN KEY (EventID) REFERENCES CAMPUS_EVENT(EventID)
        ON DELETE CASCADE
);
```

5. Sample Records

The following data represents three students, two campus organizations, two venues, three events, registrations, payments, feedback entries, and event tags. Run it after the CREATE TABLE statements.

```
INSERT INTO STUDENT VALUES
(11, 'Aarav Mehta', 'aarav.mehta@campus.edu', '9811100101', 'CSE', 3),
(12, 'Sana Iqbal', 'sana.iqbal@campus.edu', '9811100102', 'ECE', 5),
(13, 'Dev Nair', 'dev.nair@campus.edu', '9811100103', 'AI', 1);

INSERT INTO ORGANIZER VALUES
(31, 'CodeCircle Society', 'codecirlce@campus.edu'),
(32, 'Cultural Council', 'culture@campus.edu');

INSERT INTO VENUE VALUES
(41, 'Innovation Hall', 'Block C', 180),
(42, 'Open Air Stage', 'Central Lawn', 500);

INSERT INTO CAMPUS_EVENT VALUES
(501, 'Build-a-thon', '2026-11-12', '10:00:00', 150.00, 'Open', 31, 41),
(502, 'Rhythm Night', '2026-11-18', '18:30:00', 0.00, 'Planned', 32, 42),
(503, 'AI Starter Workshop', '2026-11-22', '11:00:00', 75.00, 'Open', 31, 41);

INSERT INTO REGISTRATION VALUES
(11, 501, '2026-10-09 09:15:00', 'Registered'),
(12, 501, '2026-10-09 09:22:00', 'Confirmed'),
(11, 502, '2026-10-09 09:30:00', 'Registered'),
(13, 503, '2026-10-09 09:45:00', 'Confirmed');

INSERT INTO PAYMENT VALUES
(801, 11, 501, 150.00, 'Paid', '2026-10-09'),
(802, 12, 501, 150.00, 'Pending', NULL),
(803, 13, 503, 75.00, 'Paid', '2026-10-09');

INSERT INTO FEEDBACK VALUES
(901, 11, 501, 5, 'Well-organized event and useful mentors'),
(902, 12, 501, 4, 'Good sessions; add more break time');

INSERT INTO EVENT_TAG VALUES
(501, 'Technology'), (501, 'Competition'),
(502, 'Music'), (502, 'Culture'),
(503, 'Artificial Intelligence'), (503, 'Workshop');
```

6. Retrieval and Constraint Testing

These queries demonstrate joins, aggregate functions, and foreign-key validation. The invalid registration statement is expected to fail because StudentID 999 is not present in STUDENT.

```
-- 1. List upcoming events with venue and organizer details
SELECT e.EventID, e.Title, e.EventDate, e.StartTime,
       v.VenueName, o.OrganizationName
FROM CAMPUS_EVENT AS e
LEFT JOIN VENUE AS v ON e.VenueID = v.VenueID
LEFT JOIN ORGANIZER AS o ON e.OrganizerID = o.OrganizerID
WHERE e.EventDate >= '2026-10-09'
ORDER BY e.EventDate, e.StartTime;

-- 2. Count registrations for each event
SELECT e.Title, COUNT(r.StudentID) AS RegistrationCount
FROM CAMPUS_EVENT AS e
LEFT JOIN REGISTRATION AS r ON e.EventID = r.EventID
GROUP BY e.EventID, e.Title;

-- 3. Show a student's registrations and payment status
SELECT s.FullName, e.Title, r.AttendanceStatus,
       p.Amount, p.PaymentStatus
FROM REGISTRATION AS r
JOIN STUDENT AS s ON r.StudentID = s.StudentID
JOIN CAMPUS_EVENT AS e ON r.EventID = e.EventID
LEFT JOIN PAYMENT AS p
  ON p.StudentID = r.StudentID AND p.EventID = r.EventID
WHERE s.StudentID = 11;

-- 4. Average feedback rating per event
SELECT e.Title, ROUND(AVG(f.Rating), 2) AS AverageRating
FROM CAMPUS_EVENT AS e
LEFT JOIN FEEDBACK AS f ON e.EventID = f.EventID
GROUP BY e.EventID, e.Title;

-- 5. Demonstrate a foreign-key violation (expected to fail)
-- No student with ID 999 exists.
INSERT INTO REGISTRATION
VALUES (999, 501, '2026-10-09 10:00:00', 'Registered');

-- 6. Demonstrate ON DELETE CASCADE in a test transaction.
-- Run only if you want to remove event 503 and its dependent rows.
START TRANSACTION;
DELETE FROM CAMPUS_EVENT WHERE EventID = 503;
SELECT * FROM REGISTRATION WHERE EventID = 503;
SELECT * FROM PAYMENT WHERE EventID = 503;
SELECT * FROM EVENT_TAG WHERE EventID = 503;
ROLLBACK;

-- 7. Demonstrate ON DELETE SET NULL (run in a test transaction).
START TRANSACTION;
DELETE FROM VENUE WHERE VenueID = 41;
SELECT EventID, Title, VenueID FROM CAMPUS_EVENT;
ROLLBACK;
```

7. Constraint Behaviour

• **Primary key:** uniquely identifies each row; REGISTRATION uses a two-column key.
• **UNIQUE:** avoids repeated student email addresses and organization names.
• **CHECK:** restricts semester, venue capacity, event fee, payment amount, and feedback rating to valid values (supported by modern MySQL versions).
• **ON DELETE CASCADE:** deleting an event removes its registrations and event tags; registration deletion also removes linked payment and feedback rows.
• **ON DELETE SET NULL:** deleting a venue or organizer leaves the event record but clears the corresponding foreign-key value.

8. Result

The Campus Event Management database is defined with relational tables, key constraints, sample records, useful retrieval queries, and test statements for referential integrity and deletion behaviour.

9. Conclusion

The ER model has been mapped to a relational design and implemented using MySQL syntax. Primary keys, foreign keys, uniqueness rules, value checks, and deletion actions support data consistency across students, events, registrations, payments, venues, organizers, and feedback.